Multi-Step Form: Validation & State Persistence

Problem: Build a multi-step registration form (Personal → Address → Account → Review) with per-step validation and progress that survives a page refresh.

Tech stack: React (Vite, JavaScript), the default since none was given

Time limit: 60 minutes, the default since none was given

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements.** Agree on scope before writing code.
2. **List interactions.** Every user action and what should happen. This becomes the checklist.
3. **Sketch the architecture.** Components, hooks, utils, and who talks to whom.
4. **Design state.** The smallest set of state; derive everything else.
5. **Define contracts.** Data shapes, API inputs and outputs, mocks.
6. **Trace one flow end to end** to make sure the design holds.
7. **Name the hard parts** and decide how to solve them before coding.
8. **Build in layers.** A working skeleton first, then one working feature at a time.
9. **Cover states and edge cases.** Empty, loading, error, success, weird input.
10. **Wrap up.** Trade-offs, what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	8 min
Skeleton: steps config + render one step + Next/Back	10 min
Validation (rules, touched, errors on Next)	12 min
Persistence (save, restore, clear)	10 min
Review step + submit (loading, errors)	10 min
Edge cases, polish, walkthrough	5 min

Think out loud

Say every decision and its reason as you make it: *"I'll derive errors instead of storing them so they can never go stale."* Interviewers grade the reasoning as much as the result. If you are stuck, say what you are trying and why.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- The form is split into steps: **Personal**, **Address**, **Account**, **Review**.
- A stepper shows progress: current step, completed steps, steps not reached yet.
- **Next** validates the current step. If anything is invalid, show the errors and stay on the step.
- **Back** goes to the previous step without validating and without losing data.
- Fields validate on **blur** (after the user leaves a field) and on **Next**.
- Cross-field rule: *Confirm password* must match *Password*.
- The **Review** step shows all values, with an **Edit** link per section.
- **Submit** sends the data to an API with loading, success and error states.
- **Persistence:** values and the current step survive a refresh, tab close or crash. After a successful submit the saved draft is cleared.

Non-functional requirements

Concern	What it means here
Performance	Saving on every keystroke is wasteful, so debounce writes to storage.
Scalability	Steps and fields come from a **config**, so adding a step or field is data, not new JSX.
Error handling	Corrupted storage, full storage, server errors (field-level and generic) must not crash the form.
Security	Passwords are **never** written to localStorage.
Accessibility	Labels tied to inputs, `aria-invalid`, `aria-describedby` for errors, focus the first invalid field, announce the step change.
Reusability	`useMultiStepForm(steps, initialValues)` works for any step config.
Extensibility	New validation rules are small functions; storage can swap to sessionStorage or a server draft API.

Clarifying questions to ask

1. Should the draft persist across browser sessions (localStorage) or only the current tab (sessionStorage)?
2. Are there sensitive fields that must not be stored (passwords, card numbers)?
3. Can users jump to any step from the stepper, or only to steps they have already reached?
4. When should errors appear: on change, on blur, or only after pressing Next?
5. Can the server reject specific fields (for example "username taken")? Where should that error show?
6. Should a draft expire, or be ignored if the form shape changes in a new release?

Assumptions we will build with

- localStorage, one draft per browser, key `multi-step-form:draft`, with a **version** number.
- Password fields are marked `sensitive` and are not persisted. After a refresh the user re-types them.

- The stepper allows jumping only to steps already reached. Jumping forward still requires the current step to be valid.
- Errors show after blur or after pressing Next (a "touched" model), not while typing the first character.
- The API is mocked: username `taken` returns a field error, email `fail@test.com` returns a generic server error.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Open the page with no draft	Step 1 (Personal), empty fields, no errors, Back disabled
Type in a field	Value updates; no error yet if the field was never blurred
Leave a field (blur) with an invalid value	That field's error appears
Fix the value of an errored field	Error disappears immediately (errors are derived from values)
Press **Next** with invalid fields	All errors on the step appear; focus moves to the first invalid field; step does not change
Press **Next** with a valid step	Move to the next step; stepper marks the step as reached
Press **Enter** inside a field	Same as Next (it is a real `<form>`)
Press **Back**	Previous step, values kept, no validation
Click a reached step in the stepper	Jump there (forward jumps validate the current step first)
Click an unreached step	Nothing; the button is disabled
Change Password after Confirm was filled	Confirm's "do not match" error updates live
Refresh the page mid-way	Same step and values come back, banner "We restored your saved progress"
Refresh on Review, then Submit	Passwords were not stored, so it jumps to Account and shows their errors
Click **Start over**	Draft cleared, form back to step 1
Click **Edit** on a Review section	Jump to that step
Submit with valid data	Button shows "Submitting…" and is disabled; on success the draft is cleared and a success screen shows
Submit, server says "username taken"	Jump to Account; the error is shown on the username field; editing the field clears it
Submit, server fails	Generic error banner on Review; data stays; user can retry
localStorage full or blocked	Form works normally, just without persistence
Stored draft is corrupted or from an older version	Ignored; start fresh

---

3. Design the architecture

Module	Responsibility
`config/steps.js`	The form described as data: steps, fields, types, rules, `sensitive` flag
`utils/validators.js`	Rule factories (`required` , `minLength` , `pattern` , `matches`) and `validateStep`
`utils/storage.js`	`loadDraft` / `saveDraft` / `clearDraft` with versioning and try/catch
`api/submitForm.js`	Mock API with a delay and two failure modes
`hooks/useMultiStepForm.js`	All form logic: values, step, touched, navigation, autosave, submit
`components/Stepper.jsx`	Step indicator, clickable for reached steps
`components/Field.jsx`	One input / select / checkbox with label and error, accessible
`components/Review.jsx`	Read-only summary with Edit links
`App.jsx`	Wires the hook to the UI, owns the `<form>` element

```
App
 ├── useMultiStepForm(STEPS, INITIAL_VALUES)
 │     ├── validators  (pure)
 │     ├── storage     (localStorage I/O)
 │     └── submitForm  (mock API)
 ├── Stepper
 ├── <form>
 │     ├── Field × n      (current step)
 │     └── Review         (last step)
 └── Back / Next|Submit
```

Start simple: the first version is one component with a `step` number and `if (step === 0)` blocks. As soon as validation and persistence arrive, the logic moves into the hook and the fields into config. The UI components stay "dumb": they receive values and callbacks.

---

4. Design the state / data model

State or field	Purpose
`values`	One flat object with every field of every step, e.g. `{ firstName: '', terms: false }`
`step`	Index of the visible step
`maxStep`	Furthest step reached; controls which stepper buttons are enabled
`touched`	`{ [fieldName]: true }` ; decides **when** an error is shown
`serverErrors`	Field errors returned by the API, e.g. `{ username: 'taken' }`
`status`	`'idle' | 'submitting' | 'success' | 'error'`
`submitError`	Generic server error message

State or field	Purpose
`draft`	What was loaded from storage on mount (drives the "restored" banner)

```
const [draft, setDraft] = useState(loadDraft); // read storage once
const [values, setValues] = useState(() => ({ ...initialValues, ...pick(draft?.values ?? {},
Object.keys(initialValues)) }));
const [maxStep, setMaxStep] = useState(() => clamp(draft?.maxStep, 0, lastStep));
const [step, setStep] = useState(() => clamp(draft?.step, 0, maxStep));
const [touched, setTouched] = useState({});
const [serverErrors, setServerErrors] = useState({});
const [status, setStatus] = useState('idle');
const [submitError, setSubmitError] = useState('');
```

Senior notes

- **Errors are derived, not stored.** `errors = validateStep(steps[step], values)` runs on every render. It is a few regex checks, so it is cheap, and it can never be stale. This also makes the cross-field rule (confirm password) update automatically when the password changes.
- **What is shown = errors filtered by `touched`.** "Is it invalid?" and "should I show it yet?" are two separate questions, so they are two separate pieces of data.
- `isFirst`, `isLast`, `restored` are derived from `step` and `draft`.
- **One flat `values` object** for all steps (not one object per step): the Review step and the submit payload need everything anyway, and field names are unique.
- **Ownership:** the hook owns all form state. `Field`, `Stepper` and `Review` own nothing; they render props and call callbacks. Storage is a side effect of state, never a second source of truth.
- **Lazy initial state** (`useState(fn)`) reads localStorage once on mount instead of on every render.

---

5. Design the data structures and contracts

Step config (the form as data)

```
{
  id: 'account',               // stable id: React key, analytics, URL later
  title: 'Account',
  fields: [
    { name: 'username', label: 'Username', rules: [required(), pattern(/^\w{3,15}$/, '3-15 letters,
numbers or _')] },
    { name: 'password', label: 'Password', type: 'password', sensitive: true, rules: [required(),
minLength(8)] },
    { name: 'confirmPassword', label: 'Confirm password', type: 'password', sensitive: true,
      rules: [required(), matches('password', 'Passwords do not match')] },
    { name: 'terms', label: 'I accept the terms', type: 'checkbox', rules: [required('You must accept
the terms')] },
  ],
}
```

Why: stable `id`s and `name`s are keys for React, for storage and for server errors. Rules as functions keep validation pure and testable.

Rule contract

```
// (value, allValues) => '' when valid, or the error message
const matches = (other, msg) => (v, values) => (v === values[other] ? '' : msg);
```

Stored draft (localStorage `multi-step-form:draft`)

```
{
  "version": 1,
  "savedAt": 1793615246671,
  "step": 3,
  "maxStep": 3,
  "values": { "firstName": "Ash", "email": "a@b.co", "city": "Pune", "username": "ash_k", "terms": true
}
}
```

No `password` / `confirmPassword`: they are stripped before saving.

API contract (mocked)

Request	`submitForm(payload)`; payload is `values` without `confirmPassword`
Success	`{ id: "uuid", createdAt: "ISO date" }` after about 1 s
Field error	`Error` with `{ message: "This username is already taken", field: "username" }`
Generic error	`Error("Server error. Please try again.")`

---

6. Trace the data flow

Interaction: user fills Address and presses Next, then refreshes the page.

```
User presses Next (or Enter)
   ↓
<form onSubmit> → handleSubmit → form.next()
   ↓
validateStep(steps[1], values) → {}  (no errors)
   ↓
goTo(2): setStep(2), setMaxStep(max(1, 2))
   ↓
Re-render: Stepper marks Address done, Account current; Account fields render
   ↓
Autosave effect sees step/maxStep changed → schedules saveDraft in 400 ms
   (and registers a pagehide listener with the same data)
   ↓
400 ms later: localStorage = { version:1, step:2, maxStep:2, values (no passwords) }
   ↓
User refreshes
   ↓
useState(loadDraft) → parse JSON → version matches → draft
   ↓
values = initialValues + known keys from draft; step = 2 (clamped)
```

```
   ↓
UI: Account step, previous values intact, "We restored your saved progress"
```

---

7. Important technical challenges

Challenge 1: When to show errors (touched model + derived errors)

Problem: If errors show on every change, the user sees "Invalid email" after typing the first letter. If they only show on Next, the user fills ten fields before learning the phone number was wrong. And stored errors go stale: fix the password, and "passwords do not match" stays.

Solution: Derive errors from values on every render, and show one only when the field is `touched` (blurred, or the user pressed Next). On Next, mark every field of the step as touched and focus the first invalid one.

```
const errors = { ...validateStep(steps[step], values), ...serverErrors };
const getError = (name) => (touched[name] ? errors[name] ?? '' : '');

const next = () => {
  const stepErrors = validateStep(steps[step], values);
  if (Object.keys(stepErrors).length) {
    touchStep(step);          // reveal all errors on this step
    return stepErrors;        // caller focuses the first invalid field
  }
  goTo(Math.min(step + 1, lastStep));
  return {};
};
```

Challenge 2: Persisting without losing data or leaking secrets

Problem: (a) Writing localStorage on every keystroke is wasteful. (b) Debouncing means a refresh within 400 ms loses the last keystrokes. (c) Passwords must not be saved in plain text. (d) Storage can throw (private mode, quota). (e) An old draft may not match a new form shape.

Solution: A debounced effect plus a `pagehide` flush with the same data; strip sensitive fields; wrap storage in try/catch; version the draft and only keep keys the current form knows. Skip saving an untouched form so a refresh doesn't show a pointless "restored" banner.

```
useEffect(() => {
  const dirty = maxStep > 0 || Object.keys(initialValues).some((k) => values[k] !== initialValues[k]);
  if (status === 'success' || !dirty) return;
  const data = { values: omit(values, sensitive), step, maxStep };
  const id = setTimeout(() => saveDraft(data), SAVE_DELAY);
  const flush = () => saveDraft(data);          // fast refresh / tab close
  window.addEventListener('pagehide', flush);
  return () => { clearTimeout(id); window.removeEventListener('pagehide', flush); };
}, [values, step, maxStep, status]);
```

```
export function loadDraft() {
  try {
    const draft = JSON.parse(localStorage.getItem(KEY));
    return draft?.version === VERSION ? draft : null;   // old shape → ignore
  } catch {
```

```
    return null;                                          // corrupted → start fresh
  }
}
```

Why `pagehide` and not `beforeunload`: `pagehide` also fires on mobile and with the back/forward cache, and doesn't block navigation.

Challenge 3: Restoring safely and navigating between steps

Problem: After a refresh on Review the passwords are gone, so the user could submit an incomplete form. A stale draft might say `step: 7`. A user could click the stepper to skip a step they never filled.

Solution: Clamp the restored step to `[0, maxStep]`. The stepper only enables steps `<= maxStep`, and a forward jump validates the current step first. Submit re-validates **all** steps and jumps to the first invalid one.

```
const jumpTo = (i) => {
  if (i > maxStep || i === step) return;
  if (i > step && Object.keys(next()).length) return; // can't leave forward while invalid
  setStep(i);
};

// in submit()
const invalid = steps.findIndex((s) => Object.keys(validateStep(s, values)).length);
if (invalid !== -1) { setStep(invalid); touchStep(invalid); return; }
```

Challenge 4: Submit: double clicks and server-side field errors

Problem: A double click sends two requests. The server may reject one field ("username taken") while the user is on the Review step, where that field isn't visible.

Solution: Guard with `status`, disable the button while submitting. Map a field error back to its step, show it as that field's error, and clear it as soon as the user edits the field.

```
try {
  await submitForm(omit(values, ['confirmPassword']));
  clearDraft();
  setStatus('success');
} catch (err) {
  setStatus('error');
  if (err.field) {
    setServerErrors({ [err.field]: err.message });
    touch(err.field);
    setStep(steps.findIndex((s) => s.fields.some((f) => f.name === err.field)));
  } else {
    setSubmitError(err.message);
  }
}
```

---

8. Implement in the correct order

#	Step	Result you can see
1	Vite app, `App` with a hard-coded first step and a Next button	Something renders
2	Move fields into `config/steps.js`; generic `Field` renders from config	All 4 steps render from data
3	`step` state, Next / Back, `isFirst` / `isLast`	Can click through all steps
4	`Stepper` component with current / done styling	Progress is visible
5	`validators.js` + `validateStep`; Next blocks on errors	Can't advance with bad data
6	`touched` + blur handling + focus first invalid field	Errors appear at the right time
7	Cross-field `matches` rule for confirm password	Live mismatch error
8	Extract logic into `useMultiStepForm`	App becomes thin
9	`storage.js` + lazy restore + debounced autosave + pagehide flush	Refresh keeps progress
10	Strip sensitive fields, versioning, "restored" banner, Start over	Safe, resettable persistence
11	`Review` step with Edit links; stepper jumping with `maxStep`	Full navigation
12	Mock API, submit states, server field errors, success screen	End-to-end flow

9. Cover all states and edge cases

State / edge case	Handling
Initial (no draft)	Step 1, empty values, no errors, Back disabled, later steps disabled in stepper
Restored draft	Values + step restored, banner with "Start over"
Field untouched and invalid	No error shown yet
Field touched and invalid	Error text, red border, `aria-invalid`, `aria-describedby`
Next with errors	All step errors shown, focus on first invalid field
Cross-field rule	Derived errors keep "Passwords do not match" in sync with both fields
Whitespace-only input	`required` and `minLength` trim the value
Checkbox required	`required` treats a boolean as valid only when `true`
Submitting (loading)	Button "Submitting…", Back and Submit disabled, double submit ignored
Success	Draft cleared, success screen, "Start a new form" resets
Server field error	Jump to the field's step, error on the field, cleared on edit
Generic server error	Banner with message, data kept, user can retry

State / edge case	Handling
Refresh on Review (passwords not stored)	Submit validates every step, jumps to Account
Corrupted JSON in storage	`loadDraft` returns `null`, start fresh
Old draft version / removed fields	Version check + only known keys are restored
Out-of-range `step` in storage	Clamped to `[0, maxStep]`
localStorage blocked or full	try/catch; form works without persistence
Refresh within the debounce window	`pagehide` flush saves the latest data
Untouched form	Not saved, so no empty "restored" banner
Empty step (Review has no fields)	Renders the summary instead of fields

10. Final structure

```
src/
├── api/
│   └── submitForm.js          # mock API
├── components/
│   ├── Field.jsx              # input / select / checkbox + error
│   ├── Review.jsx             # summary with Edit links
│   └── Stepper.jsx            # progress + navigation
├── config/
│   └── steps.js               # STEPS + INITIAL_VALUES
├── hooks/
│   └── useMultiStepForm.js    # all form logic
├── utils/
│   ├── storage.js             # versioned localStorage draft
│   └── validators.js          # rule factories + validateStep
├── App.jsx
├── App.css
├── index.css
└── main.jsx
```

11. Full working code

`src/config/steps.js`

```
import { required, minLength, pattern, matches } from '../utils/validators.js';

export const STEPS = [
  {
    id: 'personal',
    title: 'Personal',
    fields: [
      { name: 'firstName', label: 'First name', rules: [required(), minLength(2)] },
      { name: 'lastName', label: 'Last name', rules: [required()] },
      {
```

```
        name: 'email',
        label: 'Email',
        type: 'email',
        rules: [required(), pattern(/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Enter a valid email')],
      },
      {
        name: 'phone',
        label: 'Phone',
        type: 'tel',
        rules: [required(), pattern(/^\d{10}$/, 'Phone must be 10 digits')],
      },
    ],
  },
  {
    id: 'address',
    title: 'Address',
    fields: [
      { name: 'street', label: 'Street', rules: [required()] },
      { name: 'city', label: 'City', rules: [required()] },
      { name: 'zip', label: 'PIN code', rules: [required(), pattern(/^\d{6}$/, 'PIN code must be 6
digits')] },
      {
        name: 'country',
        label: 'Country',
        type: 'select',
        options: ['India', 'USA', 'UK', 'Germany'],
        rules: [required('Select a country')],
      },
    ],
  },
  {
    id: 'account',
    title: 'Account',
    fields: [
      {
        name: 'username',
        label: 'Username',
        rules: [required(), pattern(/^\w{3,15}$/, '3-15 letters, numbers or _')],
      },
      {
        name: 'password',
        label: 'Password',
        type: 'password',
        sensitive: true,
        rules: [
          required(),
          minLength(8),
          pattern(/(?=.*[A-Za-z])(?=.*\d)/, 'Use at least one letter and one number'),
        ],
      },
      {
        name: 'confirmPassword',
        label: 'Confirm password',
        type: 'password',
        sensitive: true,
        rules: [required(), matches('password', 'Passwords do not match')],
      },
      {
        name: 'terms',
        label: 'I accept the terms and conditions',
        type: 'checkbox',
        rules: [required('You must accept the terms')],
      },
```

```
    ],
  },
  { id: 'review', title: 'Review', fields: [] },
];

export const INITIAL_VALUES = Object.fromEntries(
  STEPS.flatMap((s) => s.fields).map((f) => [f.name, f.type === 'checkbox' ? false : '']),
);
```

`src/utils/validators.js`

```
// Each rule: (value, allValues) => error message, or '' when valid.
export const required = (msg = 'This field is required') => (v) =>
  (typeof v === 'boolean' ? v : String(v ?? '').trim()) ? '' : msg;

export const minLength = (n) => (v) =>
  String(v).trim().length >= n ? '' : `Must be at least ${n} characters`;

// Skips empty values so optional fields only validate when filled.
export const pattern = (re, msg) => (v) => (!v || re.test(v) ? '' : msg);

export const matches = (other, msg) => (v, values) => (v === values[other] ? '' : msg);

export function validateField(field, values) {
  for (const rule of field.rules ?? []) {
    const error = rule(values[field.name], values);
    if (error) return error; // first failing rule wins
  }
  return '';
}

export function validateStep(step, values) {
  const errors = {};
  for (const field of step.fields) {
    const error = validateField(field, values);
    if (error) errors[field.name] = error;
  }
  return errors;
}
```

`src/utils/storage.js`

```
const KEY = 'multi-step-form:draft';
const VERSION = 1; // bump when the form shape changes so old drafts are ignored

export function loadDraft() {
  try {
    const draft = JSON.parse(localStorage.getItem(KEY));
    return draft?.version === VERSION ? draft : null;
  } catch {
    return null; // corrupted JSON or storage blocked: start fresh
  }
}

export function saveDraft(data) {
  try {
    localStorage.setItem(KEY, JSON.stringify({ ...data, version: VERSION, savedAt: Date.now() }));
  } catch {
    // quota exceeded / private mode: the form still works, it just isn't persisted
```

```
  }
}

export function clearDraft() {
  try {
    localStorage.removeItem(KEY);
  } catch {
    // nothing to clear if storage is unavailable
  }
}
```

`src/api/submitForm.js`

```
// Mock API. username "taken" -> field error, email "fail@test.com" -> server error.
export function submitForm(payload) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (payload.username.toLowerCase() === 'taken') {
        reject(Object.assign(new Error('This username is already taken'), { field: 'username' }));
      } else if (payload.email === 'fail@test.com') {
        reject(new Error('Server error. Please try again.'));
      } else {
        resolve({ id: crypto.randomUUID(), createdAt: new Date().toISOString() });
      }
    }, 1000);
  });
}
```

`src/hooks/useMultiStepForm.js`

```
import { useEffect, useState } from 'react';
import { validateStep } from '../utils/validators';
import { loadDraft, saveDraft, clearDraft } from '../utils/storage';
import { submitForm } from '../api/submitForm';

const SAVE_DELAY = 400;

const clamp = (n, min, max) => Math.min(Math.max(Number(n) || 0, min), max);
const pick = (obj, keys) => Object.fromEntries(keys.filter((k) => k in obj).map((k) => [k, obj[k]]));
const omit = (obj, keys) => Object.fromEntries(Object.entries(obj).filter(([k]) => !keys.includes(k)));

export function useMultiStepForm(steps, initialValues) {
  const lastStep = steps.length - 1;
  const allFields = steps.flatMap((s) => s.fields);
  const sensitive = allFields.filter((f) => f.sensitive).map((f) => f.name);

  // Read storage once; only keep keys the current form still knows about.
  const [draft, setDraft] = useState(loadDraft);
  const [values, setValues] = useState(() => ({
    ...initialValues,
    ...pick(draft?.values ?? {}, Object.keys(initialValues)),
  }));
  const [maxStep, setMaxStep] = useState(() => clamp(draft?.maxStep, 0, lastStep));
  const [step, setStep] = useState(() => clamp(draft?.step, 0, maxStep));
  const [touched, setTouched] = useState({});
  const [serverErrors, setServerErrors] = useState({});
  const [status, setStatus] = useState('idle'); // idle | submitting | success | error
  const [submitError, setSubmitError] = useState('');
```

```
  // Derived, never stored: always in sync with values (incl. cross-field rules).
  const errors = { ...validateStep(steps[step], values), ...serverErrors };
  const getError = (name) => (touched[name] ? errors[name] ?? '' : '');

  // Debounced autosave + flush on pagehide so a fast refresh doesn't lose the last keystroke.
  useEffect(() => {
    const dirty = maxStep > 0 || Object.keys(initialValues).some((k) => values[k] !== initialValues[k]);
    if (status === 'success' || !dirty) return;
    const data = { values: omit(values, sensitive), step, maxStep };
    const id = setTimeout(() => saveDraft(data), SAVE_DELAY);
    const flush = () => saveDraft(data);
    window.addEventListener('pagehide', flush);
    return () => {
      clearTimeout(id);
      window.removeEventListener('pagehide', flush);
    };
  }, [values, step, maxStep, status]); // eslint-disable-line react-hooks/exhaustive-deps -- sensitive
comes from static config

  const touchStep = (i) =>
    setTouched((t) => ({ ...t, ...Object.fromEntries(steps[i].fields.map((f) => [f.name, true])) }));

  const setValue = (name, value) => {
    setValues((v) => ({ ...v, [name]: value }));
    if (serverErrors[name]) setServerErrors((e) => omit(e, [name]));
  };

  const touch = (name) => setTouched((t) => ({ ...t, [name]: true }));

  const goTo = (i) => {
    setStep(i);
    setMaxStep((m) => Math.max(m, i));
  };

  // Returns the current step's errors so the caller can focus the first invalid field.
  const next = () => {
    const stepErrors = validateStep(steps[step], values);
    if (Object.keys(stepErrors).length) {
      touchStep(step);
      return stepErrors;
    }
    goTo(Math.min(step + 1, lastStep));
    return {};
  };

  const back = () => setStep((s) => Math.max(0, s - 1));

  // Stepper click: only reached steps, and leaving forward requires the current step to be valid.
  const jumpTo = (i) => {
    if (i > maxStep || i === step) return;
    if (i > step && Object.keys(next()).length) return;
    setStep(i);
  };

  const submit = async () => {
    if (status === 'submitting') return;
    // Re-validate everything: sensitive fields are not restored after a refresh.
    const invalid = steps.findIndex((s) => Object.keys(validateStep(s, values)).length);
    if (invalid !== -1) {
      setStep(invalid);
      touchStep(invalid);
      return;
    }
```

```
    setStatus('submitting');
    setSubmitError('');
    try {
      await submitForm(omit(values, ['confirmPassword']));
      clearDraft();
      setStatus('success');
    } catch (err) {
      setStatus('error');
      if (err.field) {
        setServerErrors({ [err.field]: err.message });
        touch(err.field);
        setStep(steps.findIndex((s) => s.fields.some((f) => f.name === err.field)));
      } else {
        setSubmitError(err.message);
      }
    }
  };

  const reset = () => {
    clearDraft();
    setDraft(null);
    setValues(initialValues);
    setStep(0);
    setMaxStep(0);
    setTouched({});
    setServerErrors({});
    setSubmitError('');
    setStatus('idle');
  };

  return {
    step, maxStep, values, status, submitError,
    restored: Boolean(draft),
    isFirst: step === 0,
    isLast: step === lastStep,
    getError, setValue, touch, next, back, jumpTo, submit, reset,
  };
}
```

src/components/Field.jsx

```
export default function Field({ field, value, error, onChange, onBlur }) {
  const { name, label, type = 'text', options = [] } = field;
  const errorId = `${name}-error`;
  const a11y = {
    id: name,
    name,
    onBlur: () => onBlur(name),
    'aria-invalid': Boolean(error),
    'aria-describedby': error ? errorId : undefined,
  };
  const errorEl = error && <p id={errorId} className="field__error">{error}</p>;

  if (type === 'checkbox') {
    return (
      <div className="field field--checkbox">
        <label>
          <input type="checkbox" {...a11y} checked={value} onChange={(e) => onChange(name,
e.target.checked)} />
          {label}
        </label>
```

```
        {errorEl}
      </div>
    );
  }

  return (
    <div className="field">
      <label htmlFor={name}>{label}</label>
      {type === 'select' ? (
        <select {...a11y} value={value} onChange={(e) => onChange(name, e.target.value)}>
          <option value="">Select…</option>
          {options.map((o) => (
            <option key={o} value={o}>{o}</option>
          ))}
        </select>
      ) : (
        <input
          {...a11y}
          type={type}
          value={value}
          autoComplete={type === 'password' ? 'new-password' : undefined}
          onChange={(e) => onChange(name, e.target.value)}
        />
      )}
      {errorEl}
    </div>
  );
}
```

`src/components/Stepper.jsx`

```
export default function Stepper({ steps, current, maxStep, onSelect }) {
  return (
    <ol className="stepper">
      {steps.map((s, i) => (
        <li
          key={s.id}
          className={`stepper__item${i === current ? ' is-current' : ''}${i <= maxStep && i !== current
? ' is-done' : ''}`}
        >
          <button
            type="button"
            onClick={() => onSelect(i)}
            disabled={i > maxStep}
            aria-current={i === current ? 'step' : undefined}
          >
            <span className="stepper__dot">{i + 1}</span>
            <span className="stepper__label">{s.title}</span>
          </button>
        </li>
      ))}
    </ol>
  );
}
```

`src/components/Review.jsx`

```
const display = (field, value) => {
  if (field.type === 'password') return value ? '••••••••' : '—';
```

```
  if (field.type === 'checkbox') return value ? 'Yes' : 'No';
  return value || '—';
};

export default function Review({ steps, values, onEdit }) {
  return (
    <div className="review">
      {steps.map((s, i) =>
        s.fields.length ? (
          <section key={s.id} className="review__section">
            <header>
              <h3>{s.title}</h3>
              <button type="button" className="link" onClick={() => onEdit(i)}>Edit</button>
            </header>
            <dl>
              {s.fields.map((f) => (
                <div key={f.name}>
                  <dt>{f.label}</dt>
                  <dd>{display(f, values[f.name])}</dd>
                </div>
              ))}
            </dl>
          </section>
        ) : null,
      )}
    </div>
  );
}
```

`src/App.jsx`

```
import { STEPS, INITIAL_VALUES } from './config/steps';
import { useMultiStepForm } from './hooks/useMultiStepForm';
import Stepper from './components/Stepper';
import Field from './components/Field';
import Review from './components/Review';
import './App.css';

export default function App() {
  const form = useMultiStepForm(STEPS, INITIAL_VALUES);
  const { step, values, status } = form;
  const current = STEPS[step];
  const submitting = status === 'submitting';

  const handleSubmit = (e) => {
    e.preventDefault();
    if (form.isLast) return form.submit();
    const errors = form.next();
    const firstInvalid = current.fields.find((f) => errors[f.name]);
    if (firstInvalid) document.getElementById(firstInvalid.name)?.focus();
  };

  if (status === 'success') {
    return (
      <main className="app">
        <div className="card success">
          <h1>Registration complete</h1>
          <p>Thanks, {values.firstName}! Your account has been created.</p>
          <button type="button" className="btn btn--primary" onClick={form.reset}>Start a new
form</button>
        </div>
```

```
      </main>
    );
  }

  return (
    <main className="app">
      <div className="card">
        <h1>Create your account</h1>
        <Stepper steps={STEPS} current={step} maxStep={form.maxStep} onSelect={form.jumpTo} />
        <p className="sr-only" aria-live="polite">
          Step {step + 1} of {STEPS.length}: {current.title}
        </p>

        {form.restored && (
          <div className="banner">
            We restored your saved progress.
            <button type="button" className="link" onClick={form.reset}>Start over</button>
          </div>
        )}

        <form onSubmit={handleSubmit} noValidate>
          <h2>{current.title}</h2>
          {form.isLast ? (
            <Review steps={STEPS} values={values} onEdit={form.jumpTo} />
          ) : (
            current.fields.map((f) => (
              <Field
                key={f.name}
                field={f}
                value={values[f.name]}
                error={form.getError(f.name)}
                onChange={form.setValue}
                onBlur={form.touch}
              />
            ))
          )}

          {form.submitError && <p className="form-error" role="alert">{form.submitError}</p>}

          <div className="actions">
            <button type="button" className="btn" onClick={form.back} disabled={form.isFirst ||
submitting}>
              Back
            </button>
            <button type="submit" className="btn btn--primary" disabled={submitting}>
              {form.isLast ? (submitting ? 'Submitting…' : 'Submit') : 'Next'}
            </button>
          </div>
        </form>
      </div>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

`src/App.css`

```
.app { display: flex; justify-content: center; padding: 40px 16px; }
.card { width: 100%; max-width: 560px; background: #fff; border-radius: 12px; padding: 28px; box-shadow:
0 4px 24px rgb(0 0 0 / 8%); }
.card h1 { margin: 0 0 20px; font-size: 1.5rem; }
.card h2 { margin: 8px 0 16px; font-size: 1.15rem; }

.stepper { display: flex; list-style: none; padding: 0; margin: 0 0 20px; gap: 4px; }
.stepper__item { flex: 1; }
.stepper__item button { width: 100%; display: flex; flex-direction: column; align-items: center; gap:
6px; background: none; border: 0; cursor: pointer; color: #666; font: inherit; }
.stepper__item button:disabled { cursor: not-allowed; opacity: 0.5; }
.stepper__dot { width: 30px; height: 30px; border-radius: 50%; display: grid; place-items: center;
border: 2px solid #ccc; font-weight: 600; }
.stepper__label { font-size: 0.8rem; }
.is-done .stepper__dot { border-color: #16a34a; color: #16a34a; }
.is-current .stepper__dot { background: #2563eb; border-color: #2563eb; color: #fff; }
.is-current .stepper__label { color: #2563eb; font-weight: 600; }

.banner { display: flex; justify-content: space-between; align-items: center; background: #eff6ff;
color: #1e40af; padding: 10px 12px; border-radius: 8px; font-size: 0.9rem; margin-bottom: 12px; }

.field { display: flex; flex-direction: column; gap: 4px; margin-bottom: 14px; }
.field label { font-size: 0.9rem; font-weight: 500; }
.field input:not([type='checkbox']), .field select { padding: 10px 12px; border: 1px solid #ccc; border-
radius: 8px; font: inherit; }
.field input:focus, .field select:focus { outline: 2px solid #2563eb; outline-offset: 1px; }
.field input[aria-invalid='true'], .field select[aria-invalid='true'] { border-color: #dc2626; }
.field--checkbox label { display: flex; gap: 8px; align-items: center; font-weight: 400; }
.field__error { margin: 0; color: #dc2626; font-size: 0.8rem; }

.review__section { border: 1px solid #e5e7eb; border-radius: 8px; padding: 12px 14px; margin-bottom:
12px; }
.review__section header { display: flex; justify-content: space-between; align-items: center; }
.review__section h3 { margin: 0; font-size: 1rem; }
.review dl { margin: 8px 0 0; display: grid; gap: 4px; }
.review dl div { display: flex; justify-content: space-between; gap: 12px; font-size: 0.9rem; }
.review dt { color: #666; }
.review dd { margin: 0; text-align: right; word-break: break-word; }

.form-error { color: #dc2626; background: #fef2f2; padding: 10px 12px; border-radius: 8px; }
.actions { display: flex; justify-content: space-between; margin-top: 20px; }
.btn { padding: 10px 20px; border-radius: 8px; border: 1px solid #ccc; background: #fff; font: inherit;
cursor: pointer; }
.btn:disabled { opacity: 0.5; cursor: not-allowed; }
.btn--primary { background: #2563eb; border-color: #2563eb; color: #fff; }
.link { background: none; border: 0; color: #2563eb; cursor: pointer; font: inherit; text-decoration:
underline; }
.success { text-align: center; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-
space: nowrap; }
```

`src/index.css`

```
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif; background:
#f3f4f6; color: #111827; }
```

---

12. Wrap-up

Edge cases handled

- Validation on blur and on Next, focus on the first invalid field, live cross-field validation.
- Persistence with debounce **and** a `pagehide` flush, so a quick refresh loses nothing.
- Passwords never written to storage; submit re-validates every step to catch them after a refresh.
- Corrupted, outdated or partially unknown drafts are ignored or filtered; storage errors never break the form.
- Double submit blocked; server field errors routed to the right step and field.
- Untouched forms aren't saved; "Start over" clears both storage and state.

Trade-offs to mention

- **localStorage vs sessionStorage vs server drafts:** localStorage survives closing the browser but is per device and readable by any script on the origin (XSS). Server drafts work across devices but need auth and an API.
- **Derived errors on every render:** simple and always correct; fine for dozens of fields. For hundreds of fields or async rules, validate only the changed field and memoize.
- **Own hook vs a library:** React Hook Form + Zod would give uncontrolled inputs (fewer re-renders) and schema validation. Writing it by hand shows the fundamentals in an interview.
- **Controlled inputs:** every keystroke re-renders the step. That's fine at this size; uncontrolled inputs scale better.
- **Not persisting passwords** costs the user a little re-typing, in exchange for not leaking secrets.
- **Flat values object:** simple, but field names must be unique across steps.

What to add with more time

- **Tests:** unit tests for validators and storage (already pure); React Testing Library for Next/Back, restore after remount, server error routing.
- **Async validation** (username availability) with debounce and AbortController.
- **Multi-tab sync** with the `storage` event (or warn "this form is open in another tab").
- **Draft expiry** using `savedAt` (e.g. discard after 7 days).
- **URL per step** (`?step=address`) so Back/Forward in the browser works.
- **Focus management** on step change (move focus to the step heading).
- **Conditional steps** (e.g. skip Address for some users) via a `when(values)` function in the config.
- **Schema validation** with Zod shared with the backend.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why localStorage and not sessionStorage?

The requirement is "survive refresh and accidental close". sessionStorage dies with the tab, so closing the tab loses everything. If the data were more sensitive, or drafts shouldn't outlive the tab, I'd switch to sessionStorage; it's a one-line change in `storage.js`.

Q2. What should never be persisted, and how do you enforce it?

Passwords, card numbers, OTPs, government IDs. Fields carry a `sensitive: true` flag in the config and the autosave strips them with `omit(values, sensitive)`. The rule lives in data, so a new sensitive field can't be forgotten in the save code.

Q3. What if the user opens the form in two tabs?

Both tabs write the same key and the last write wins. With more time I'd listen to the `storage` event to sync or show "this form is open in another tab", or add a tab id plus timestamp and keep the newest.

State design

Q4. Why not store `errors` in state?

Stored errors must be updated on every change and can go stale (fix the password, the "do not match" error stays on confirm). Derived errors are always consistent with values. State holds only what can't be computed: `touched`.

Q5. Why one flat `values` object instead of one object per step?

Review and submit need all values together, field names are unique, and updates are a single `{ ...v, [name]: value }`. Nested per-step objects add path handling for no gain here.

Q6. Why `useState(loadDraft)` instead of loading in a `useEffect`?

The lazy initializer runs once, before the first paint, so the restored values render immediately. An effect would render an empty form first and then flash to the restored one, and could even save the empty form over the draft.

Q7. Would you use `useReducer`?

It's a good fit once there are many related transitions (next, back, jump, submit start/success/fail). I'd move to a reducer when the hook grows; it also makes the transitions unit-testable without React.

Validation

Q8. Why validate on blur and not on change?

On change, the user sees "invalid email" after the first letter, which is noisy. On blur, they get feedback when they're done with the field. After an error is shown, it clears live on change because errors are derived.

Q9. How would you add async validation, like "username available"?

A debounced check (about 400 ms) on change, with an AbortController to cancel stale requests, a `checking` state, and the result in a separate `asyncErrors` map. Next waits for pending checks. The server still validates on submit, since the client check is only UX.

Q10. Is client-side validation enough?

No. It's for UX only. Anyone can bypass it, so the server must validate everything again; that's why the submit handles server field errors.

Persistence

Q11. Why debounce the save, and what about losing the last keystrokes?

Debouncing avoids serializing and writing on every key. To avoid losing the last 400 ms, the same effect registers a `pagehide` listener that saves immediately. `pagehide` is more reliable than `beforeunload` on mobile and with the back/forward cache.

Q12. What if you release a new version of the form with different fields?

The draft has a `version`; bump it and old drafts are ignored. For a softer path, restore only keys the current form knows (`pick(draft.values, Object.keys(initialValues))`), or write a migration from v1 to v2.

Q13. What if localStorage throws?

In Safari private mode, or when the quota is exceeded, `setItem` throws. All storage calls are wrapped in try/catch so the form keeps working, just without persistence. `JSON.parse` of corrupted data is caught the same way.

Q14. What if the data is huge, e.g. file uploads in a step?

Don't put files in localStorage (about 5 MB limit, strings only). Upload the file right away and store only the returned id or URL, or use IndexedDB for blobs.

Navigation and submit

Q15. Why can't the user click any step in the stepper?

Skipping steps leads to submitting incomplete data and confusing error jumps. Only reached steps are clickable, a forward jump validates the current step, and submit re-validates all steps as the final guard.

Q16. How do you prevent double submission?

`status === 'submitting'` makes `submit()` return early, and the button is disabled. On the server, an idempotency key (for example the draft id) is the real guarantee.

Q17. What if the API is slow or times out?

Show the loading state, keep the data (the draft is only cleared on success), and on timeout show the generic error with a retry. I'd add an AbortController with a timeout, and maybe keep the draft id so a retry is idempotent.

Q18. The server rejects a field while the user is on Review. How does the user find it?

The error has a `field`; we find the step that contains that field, jump there, mark it touched and show the message on the field itself. Editing the field removes the server error.

Performance and scale

Q19. What if the form had 200 fields?

Controlled inputs re-render the whole step on each keystroke, and every step is validated on each render. I'd memoize `Field` with `React.memo` and stable callbacks, validate only the changed field, or switch to uncontrolled inputs (React Hook Form). Also split long steps.

Q20. Why not just use Formik / React Hook Form + Zod?

In production I probably would: less code, fewer re-renders, a shared schema with the backend. In an interview, building it shows I understand touched vs dirty, derived errors and persistence. The hook's API is close to those libraries, so swapping later is easy.

Accessibility

Q21. How is this form accessible?

Every input has a `<label htmlFor>`. Errors are linked with `aria-describedby` and flagged with `aria-invalid`. Focus moves to the first invalid field on Next. The current step uses `aria-current="step"`, and a polite live region announces "Step 2 of 4: Address". Enter submits the step because it's a real `<form>`.